



Energy Intelligence Platform

Deployment and CI/CD Guide

Version 1.0

July 30, 2026

Team Coreflow



COS 301 Capstone Project

University of Pretoria

OptiGrid CI/CD Pipeline

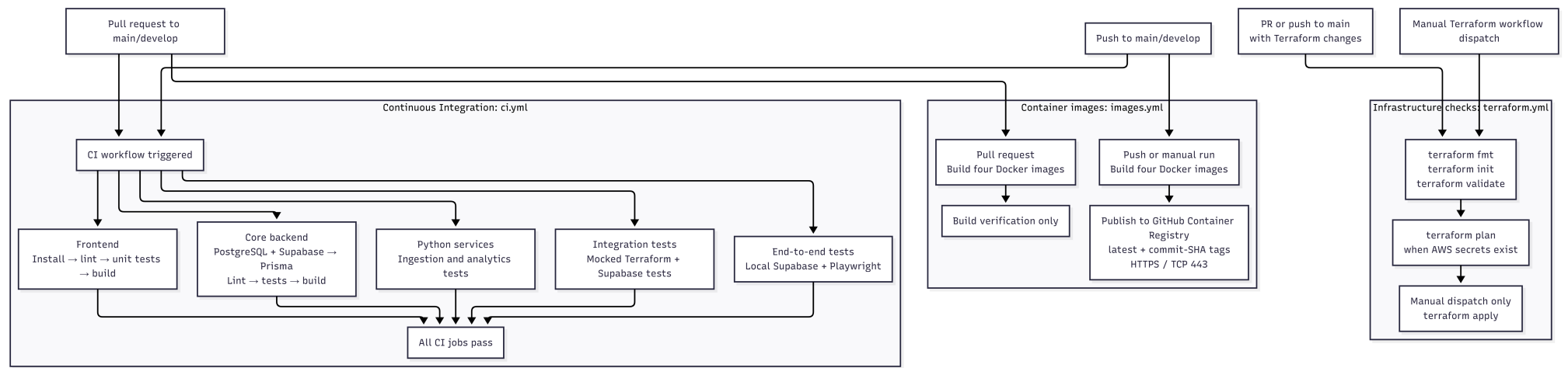


Figure 1: OptiGrid Continuous Integration, Docker image publishing, and Terraform infrastructure workflow.

CI/CD Pipeline Explanation

The OptiGrid CI/CD process begins when a pull request is opened against `main` or `develop`, or when code is pushed to either branch. The Continuous Integration workflow validates all major components of the system before changes are merged.

- **Frontend CI:** Installs dependencies, runs linting and unit tests, and builds the Next.js frontend application.
- **Core Backend CI:** Starts PostgreSQL and local Supabase, applies the Prisma schema, and then runs backend linting, unit tests, and the TypeScript build.
- **Python Services CI:** Runs unit tests for the ingestion and analytics services.
- **Integration and E2E Testing:** Runs mocked Terraform tests, Supabase-backed integration tests, and Playwright end-to-end tests.
- **Docker Image Publishing:** Pull requests only verify that the frontend, core, ingestion, and analytics Docker images can build. Pushes publish images to GitHub Container Registry using both `latest` and `commit-SHA` tags over HTTPS on TCP port 443.
- **Terraform Workflow:** Terraform formatting, initialisation, validation, and planning are performed for infrastructure changes. Infrastructure creation or modification using `terraform apply` only occurs through a manual workflow dispatch.

Although Docker images are automatically published after a push, OptiGrid does not currently use automatic Continuous Deployment. Therefore, a successful CI pipeline does not automatically deploy the application to AWS.

OptiGrid Manual Deployment Pipeline

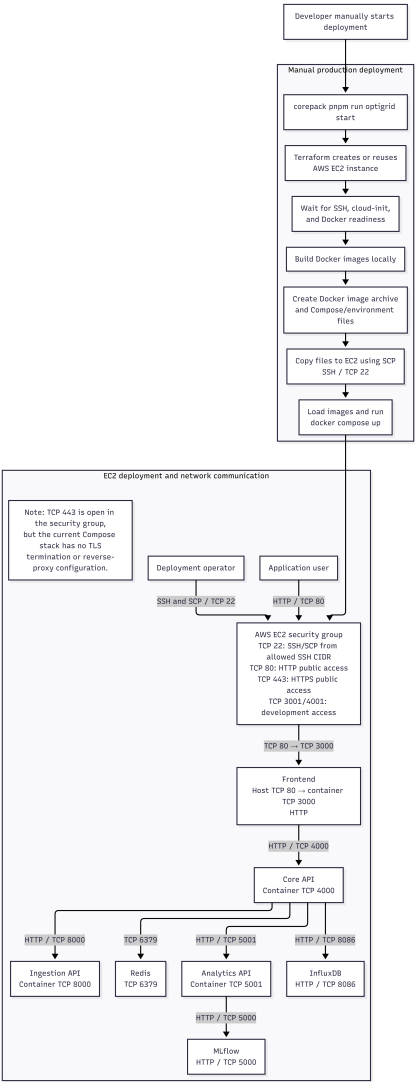


Figure 2: Manual EC2 deployment process, exposed ports, protocols, and internal Docker service communication.

Manual Deployment Pipeline Explanation

Production deployment is started manually by a developer using:

```
corepack pnpm run optigrid start
```

The deployment script performs the following steps:

1. Terraform creates a new AWS EC2 instance or reuses the existing instance recorded in the Terraform state.
2. The script waits until SSH access, cloud-init, and Docker are ready on the EC2 instance.
3. The frontend, core backend, ingestion, and analytics Docker images are built locally. The Redis and InfluxDB dependency images are included in the deployment archive.
4. Docker images, Docker Compose configuration, and environment files are copied to the EC2 instance using Secure Copy Protocol (SCP), which runs over SSH on TCP port 22.
5. The EC2 instance loads the Docker image archive and starts all containers using `docker compose up`.

The frontend is publicly available through HTTP on TCP port 80. Docker maps this host port to the frontend container on TCP port 3000. The frontend then communicates with the core API through the internal Docker network using HTTP on TCP port 4000.

The core API communicates internally with the ingestion API on TCP port 8000, the analytics API on TCP port 5001, Redis on TCP port 6379, and InfluxDB on TCP port 8086. The analytics service communicates with MLflow using HTTP on TCP port 5000.

The AWS security group allows SSH/SCP access on TCP port 22 from the configured `allowed_ssh_cidr`, public HTTP access on TCP port 80, and public HTTPS access on TCP port 443. Ports 3001 and 4001 are available for the development frontend and development core API respectively.

Security note: TCP port 443 is open in the current security group; however, the current Docker Compose configuration does not include a reverse proxy or TLS certificate configuration. HTTPS should therefore be configured through a reverse proxy, load balancer, or similar TLS termination mechanism before production use.